The complete workflow for scaling-law experiments

> **Note**
>
> For the latest conversation evidence and **96-GPU × 30-day** allocation, see the industry audit, data-engineering companion, trainer playbook, checkpoint evaluation, and current numerical plan. Earlier 128×40 examples remain historical alternatives.

A scaling ladder reduces uncertainty about a specific target training decision. Start with the decision, design small experiments that identify the required effects, and test the resulting forecast on runs that were excluded from selection. A smooth fitted curve is one intermediate artifact.

The source blog is a synthesis of public work. This guide organizes that synthesis into an executable research plan; it does not report new training runs or endorse universal hyperparameter values.

For the current architecture questions, begin with concrete width/depth and μP evidence, then consult the review of the supplied notes.

For a detailed plan starting with available data, read from Nemotron-CC or Marin data to a validated model, with separate empirical accounts of OLMo and Marin.

Workflow and decision gates

Step	Work	Artifact	Gate to proceed
1	Define target and objective	Decision statement	Metric, budget, constraints explicit
2	Freeze measurements and evaluation	Measurement contract	Comparable parameters, tokens, loss, FLOPs
3	Establish a faithful baseline	Versioned recipe	Stable pilots and measured throughput
4	Screen data, structure, optimizers separately	Controlled ablations	Effects exceed uncertainty
5	Design size × horizon experiments	Run matrix	Both axes identifiable; holdouts reserved
6	Tune or prescribe hyperparameters	Tuning ledger or scaling heuristic	Policy matches claimed objective
7	Run the ladder and collect endpoints	Run/branch lineage	Annealing and stage conventions match
8	Fit candidate surfaces	Residuals and decision intervals	Development prediction is acceptable
9	Forecast downstream tasks	Task calibrations	Continuous signal and independent checks
10	Validate combined production recipe	Holdout/stability report	All preregistered gates pass
11	Archive and monitor transfer	Reproducible experiment bundle	Recipe changes trigger a transfer audit

A concrete planning example

Suppose the decision is to choose a dense model and token horizon under training compute C_*, with a fixed context length and finite corpus. Use the following as an illustrative design, not a published budget prescription.

1. Inventory unique tokens U_j by bucket and specify a common validation distribution. Set domain capability floors.
2. Pilot a small baseline to measure throughput, seed noise, and a usable LR/batch region.

3. Compare a few data and architecture hypotheses at matched budgets. Confirm promising effects at a second size.
4. Select three fitting sizes and three horizons that bracket the intended target TPP. Reserve a larger size and a longer-horizon configuration. Give each candidate comparable tuning effort when estimating a tuned frontier.
5. Train and anneal all comparable endpoints. Record shared-prefix branches and failed runs.
6. Fit an additive loss surface and a justified alternative. Hold out development slices in the extrapolation direction; inspect residuals rather than selecting by training error.
7. Solve the constrained allocation problem and bootstrap the complete procedure. If a budget constraint is $C(N, D) \leq C_*$, the target is

$$(N_*, D_*) = \arg\min_{N,D} \widehat{L}(N, D) \quad \text{subject to compute, memory, data-exposure, and serving constraints.}$$

8. Compare the chosen configuration against neighboring allocations on fresh holdouts. Re-run the complete candidate stack at intermediate scale, using production precision and parallelism.
9. Commit to the flagship only when prediction, capability, stability, and systems gates pass. If the holdout changes the recipe, reserve a new test.

What to keep fixed, and what to learn

A fixed-recipe forecast asks how a declared heuristic behaves. A frontier experiment asks what can be achieved after tuning each point. Do not fit under-tuned small runs and label the resulting optimum a compute-optimal frontier. Likewise, a mixture experiment needs fixed validation data; its own training loss is not a fair quality comparator.

Allocate uncertainty to the actual decision: candidate margin, allocation regret, epoch exposure, or wall-clock budget. A narrow coefficient interval is insufficient if the recommended model size changes sharply across plausible fits.

Separate explanations of every source section

1. Scope: what a scaling ladder predicts
2. Objectives, evaluation, and acceptance criteria
3. Measurement: parameters, FLOPs, tokens, and loss
4. Architecture: a controlled baseline and structural ladders
5. Experimental grid, checkpoints, and compute budget
6. Optimizers, hyperparameter transfer, and annealed comparisons
7. Data experiments: source value, mixture weights, and repetition
8. Fitting loss surfaces and diagnosing extrapolation
9. Predicting downstream capabilities
10. Holdouts, stability, and production decisions
11. Staged training and long-context ladders
12. Execution records, deviations, and ladder maintenance
13. Public examples: Chinchilla, optimizer grids, and Delphi

Minimum deliverable bundle

Keep the experiment objective, measurement contract, dataset manifests, complete run matrix, tuning budgets, branch/seed lineage, fit scripts and residuals, bootstrap decisions, downstream calibration, holdout outcomes, and a production acceptance record together. Record what failed and what evidence led to each revision.

Sources: Jiaxuan Zou, source overview; Chinchilla; Porian et al.; OLMo Task Ladder; Marin Delphi. The separate guides provide topic-specific primary citations.